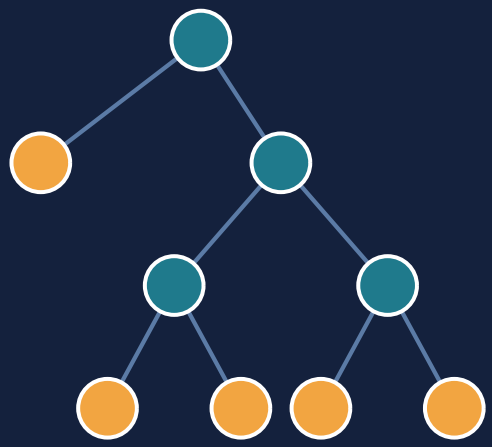
TECHNICAL REPORT · C++17 · LOSSLESS COMPRESSION

FileCompressor

Huffman Coding, from Intuition to Implementation



A visual, first-principles guide to how the compressor works, why it works, how well it performs, and what remains to verify.

Formats covered: HUF1 (legacy) · HUF2 (under verification) Edition: 11 October 2026

ABSTRACT

FileCompressor is a C++17 command-line program that compresses and restores files without losing a single byte. It treats every file as a stream of bytes, counts how often each byte value occurs, and assigns shorter bit patterns to common bytes and longer ones to rare bytes using **Huffman coding**. This report builds that idea from first principles: why variable-length codes save space, how the code tree is constructed and stored, how bits are packed into bytes, how the HUF1 and HUF2 file formats are laid out, and what the recorded benchmarks show on 100 MiB to 1 GiB inputs. It also states plainly what is *not yet proven*: the HUF2 raw-mode path has shown a byte mismatch in a random-binary round-trip test and must be treated as unverified.

KEYWORDS lossless compression · Huffman coding · prefix codes · entropy · bit-level I/O · file format design · C++17

256	2,060 B	43.05%	O(n)
possible byte values; one frequency slot each	fixed HUF1 header (4 + 8 + 256 × 8)	space saved on the 100 MiB benchmark file	time: a linear pass over the data

READING GUIDE

If you want to…	Read
get the core idea in five minutes	Sections 1–3 and Figures 1–4
understand exactly how the bits and bytes are laid out	Sections 4 and 6 (Figures 5, 6, 8)
see how the code is organised and how data flows	Sections 5 and 7 (Figures 7, 9)
judge performance and limits	Sections 8–9 (Figures 10–12)
know what is verified and what is not	Section 10
test yourself	Section 12 (exercises and glossary)

Contents

1 The core intuition . . . 3

2 How much can we hope to save? . . . 3

3 Building the Huffman tree . . . 4

4 Prefix codes and bit-level I/O . . . 6

5 System architecture . . . 8

6 The compressed file format . . . 9

7 Compression and decompression pipelines . . . 10

8 Complexity analysis . . . 11

9 Empirical results . . . 12

10 Verification status and the open issue . . . 13

11 Limitations and the road ahead . . . 15

12 Study guide . . . 15

A Appendix: pseudocode and commands . . . 17

NOTE

A note on evidence. Everything labelled *recorded* comes from the project's own documentation (benchmarks, file-format sizes, test status). Everything labelled *model*, *illustration* or *suggestion* is explanatory material added in this report to build intuition; it is not a measurement of your program.

1 The core intuition

A computer stores a file as bytes, and a byte can hold 256 different values. A plain file spends exactly **8 bits on every byte**, whether that byte is a very common one (a space in English text, a zero in a binary table) or one that appears once in a million. That is wasteful in the same way it would be wasteful for a postal service to give every street address the same long code, including the busiest ones.

Compression by Huffman coding fixes this with one idea: **spend fewer bits on what you say often, and more bits on what you say rarely**. Morse code works the same way: the most common English letter, E, is a single dot, while rare letters take four signals. If the savings on the frequent symbols outweigh the extra cost on the rare ones, the whole message gets shorter.

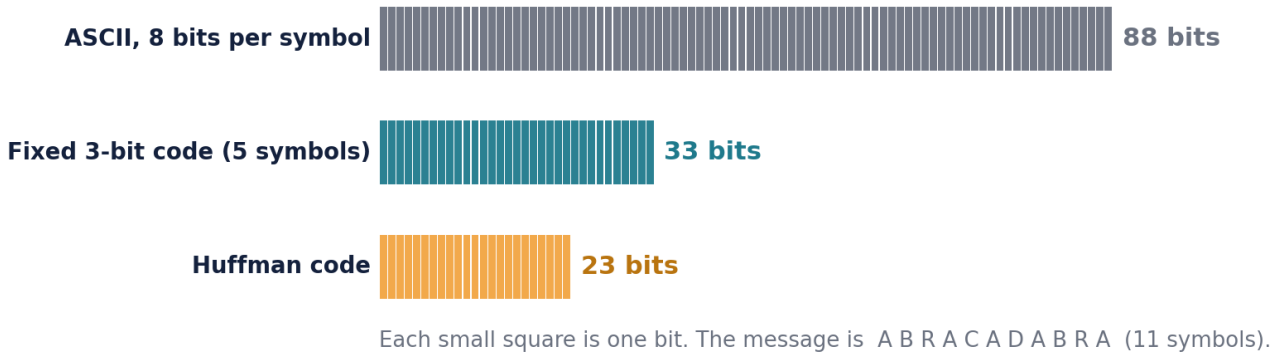


Figure 1. The same 11-symbol message stored three ways. Plain bytes need 88 bits; a fixed 3-bit code (enough for 5 distinct symbols) needs 33; a Huffman code gives the common symbol A a one-bit code and needs only 23. *Illustration.*

> **INTUITION**
> Think of a Huffman code as a **custom shorthand for one particular file**. The compressor first reads the whole file to learn which bytes are common, then invents a shorthand that makes *that* file small. The decoder is given enough information to recreate the same shorthand, and translates back.

To make this work, three questions need answers. This report is organised around them:

- **Which symbols get short codes, and how short?** Answered by the Huffman tree (Section 3).
- **How does the decoder know where one code ends and the next begins**, when no separators are stored? Answered by the prefix property (Section 4).
- **What must be saved in the file** so the decoder can rebuild the same tree? Answered by the file header (Section 6).

Because the output is lossless, decoding must return the original byte stream exactly. Anything that changes even one byte, even in a path that otherwise 'works', is a defect, which is why Section 10 treats verification so seriously.

2 How much can we hope to save?

Before building the tree it helps to know the limit. Claude Shannon's **entropy** measures the average information per symbol, in bits, given how often each symbol occurs:

$$H = -\Sigma\, p_i \log_2 p_i$$

Here p_i is the probability (relative frequency) of symbol i. For a code that assigns each symbol its own whole number of bits, entropy is the best possible average. Huffman coding is *optimal* among such codes, and its average length L always lands within one bit of the limit: **H ≤ L < H + 1**.

For the example message of Section 1, the probabilities are 5/11, 2/11, 2/11, 1/11 and 1/11, giving H ≈ 2.04 bits per symbol, while the Huffman code actually spends 23/11 ≈ 2.09. The code is 0.05 bits per symbol away from the theoretical floor on this tiny example.

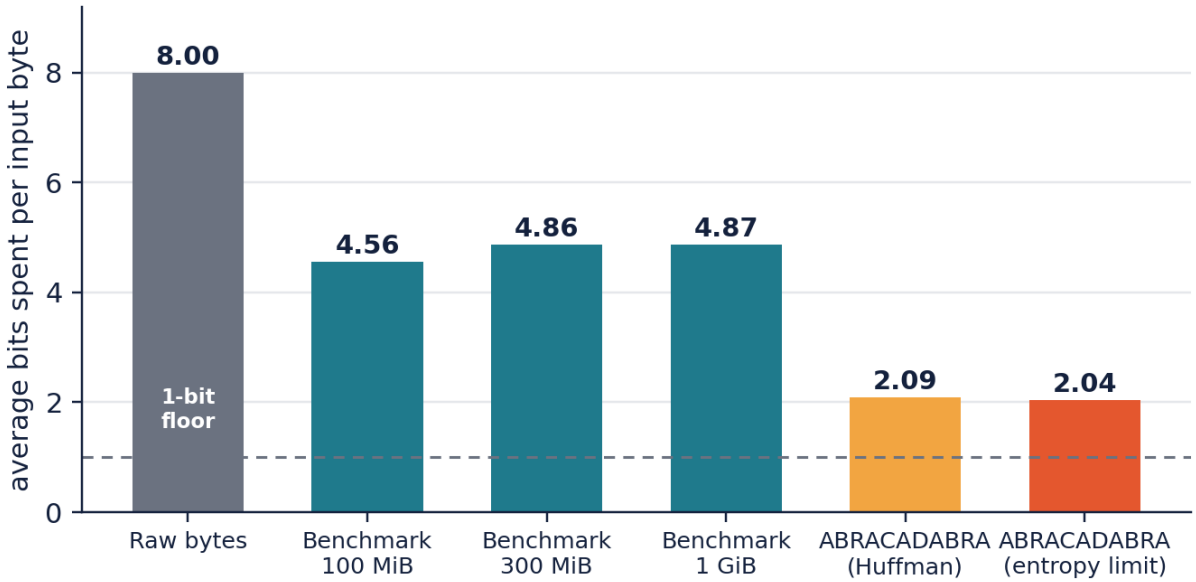


Figure 2. Average bits spent per input byte. Benchmark bars are derived from the *recorded* size ratios (ratio × 8 bits) and are the whole compressed file, header included. The two ABRACADABRA bars are the worked example computed in this report. Bars come from different data, so compare each to the 8-bit baseline, not to each other.

KEY IDEA

The compression ratio is decided by the data, not by the program. A file whose bytes are very uneven (text, logs, sparse tables) has low entropy and shrinks a lot. A file whose bytes are nearly uniform (encrypted, random or already-compressed data) has entropy near 8 bits and cannot shrink; the header makes it slightly *bigger*.

WATCH OUT

Byte-wise Huffman coding cannot spend less than **one whole bit per symbol**. Even a file of 1,000,000 identical bytes (entropy 0) needs at least 1,000,000 bits ≈ 125,000 bytes with this scheme, so the best pure byte-level saving is 87.5%, before header overhead.

3 Building the Huffman tree

A Huffman code is stored as a **binary tree**. Every byte that occurs becomes a leaf; its code is the sequence of left/right turns (0 or 1) from the root down to that leaf. Frequent bytes should sit near the root (short path) and rare bytes deep in the tree (long path). The construction below, taken from the project's design, does exactly that:

Table 1. Huffman tree construction (recorded design) with the reasoning behind each step.

#	Step	Why
1	Create a leaf for every byte whose frequency is non-zero.	Unused byte values need no code.
2	Put the leaves in a min-priority queue keyed by frequency.	Lets us repeatedly grab the two smallest in O(log k).
3	Remove the two nodes with the smallest frequencies.	The two rarest symbols deserve the two longest codes.
4	Create a parent whose frequency is their sum; the two nodes become its children.	The parent stands in for 'either of these'.
5	Insert the parent back into the queue.	It competes in the next round like any node.
6	Repeat until one node remains: the root.	k leaves need exactly k − 1 merges.

Figure 3 walks through the construction on the message ABRACADABRA, whose byte counts are A=5, B=2, R=2, C=1, D=1. Each step merges the two lightest trees, and the red ring marks the node that was just created.

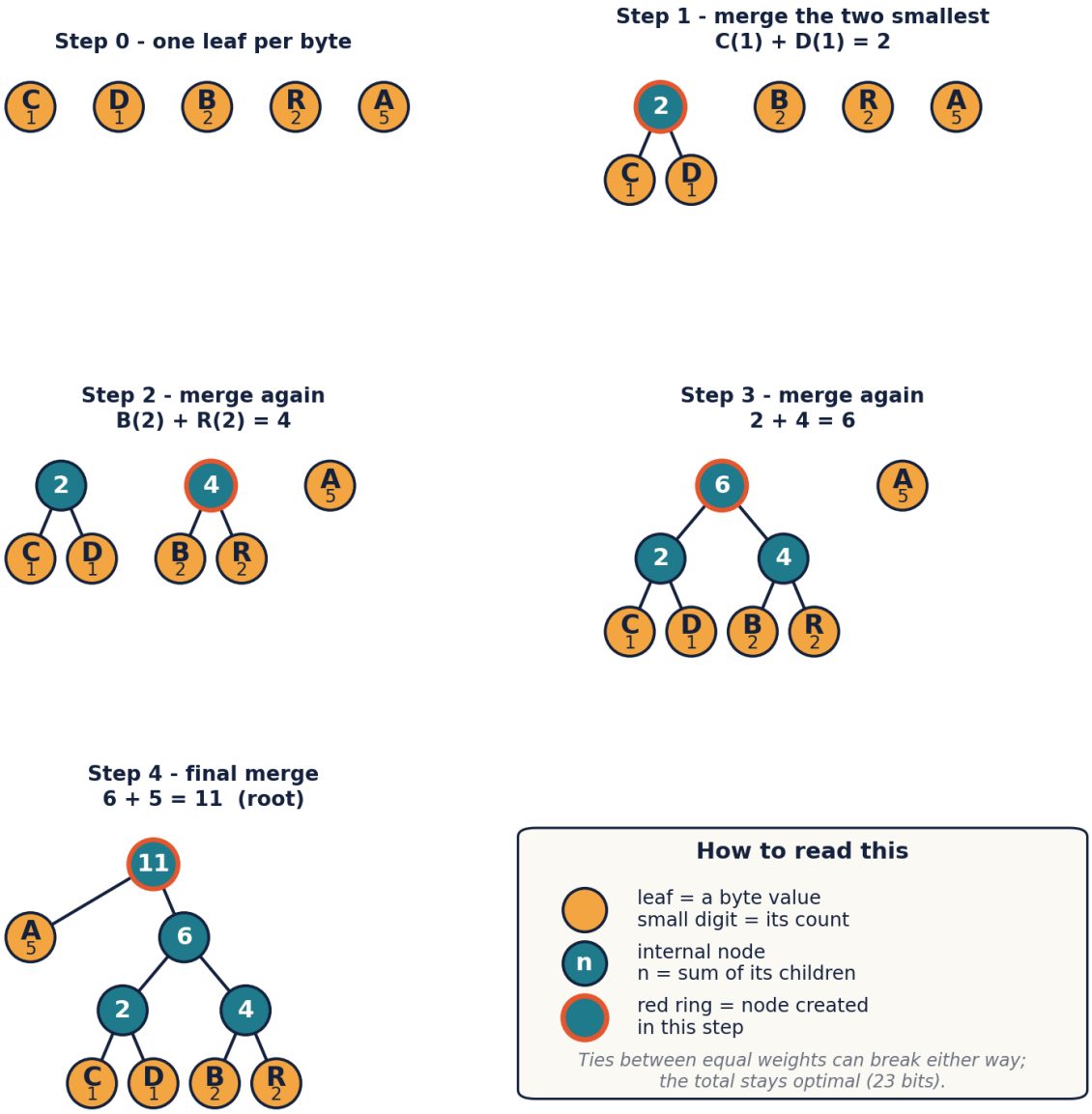
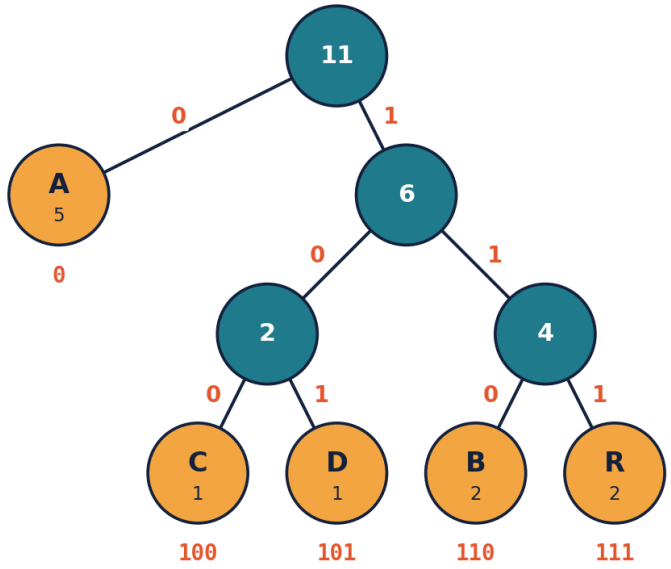


Figure 3. Building the tree for ABRACADABRA. Weights add up to 11, the message length. When several nodes tie (for example three nodes of weight 2) the tie-break may pick differently; the project uses a deterministic rule so the same frequencies always give the same tree. *Illustration; tie-breaks shown here are not necessarily the project's.*

Reading the finished tree gives the code book. Every left edge is a 0 and every right edge a 1, so the code of a leaf is the path to it.



Code book

Byte	Count	Code	Bits
A	5	0	5x1=5
B	2	110	2x3=6
R	2	111	2x3=6
C	1	100	1x3=3
D	1	101	1x3=3

Total = 23 bits (avg 2.09 bits/symbol)

Figure 4. The final tree and its code book. A, the most common byte, receives the 1-bit code 0; the two rarest bytes receive 3-bit codes. The total, 23 bits, is shown as count × code length per byte.

Why merging the two smallest is the right move

The two least frequent symbols belong at the very bottom of the tree, as siblings, since any other placement could be swapped to shorten the total. Merging them into one super-symbol and solving the smaller problem repeats that argument one level up. This 'locally best choice is globally best' structure is why Huffman's algorithm (published in 1952) is a textbook *greedy algorithm*. A different tie-break can yield a different tree, but every valid tree has the same minimum total length; here, always 23 bits.

> WATCH OUT
>
> **Two special cases need explicit handling.** With only *one* distinct byte value the tree is a single leaf and has no left/right choices, so the implementation assigns that symbol the 1-bit code `0`; the decoder must treat this case explicitly. With an *empty* input the original size is zero and the decoder must produce an empty output without trying to decode any symbol.

4 Prefix codes and bit-level I/O

4.1 Why no separators are needed

Look at the code book again: `0`, `100`, `101`, `110`, `111`. No code is the beginning of another code. This is the **prefix property**, and it is automatic in a Huffman tree because symbols live only at the leaves: you can never pass through one symbol on the way to another. So the decoder simply walks down the tree, one bit per edge, and the moment it lands on a leaf it knows a symbol is complete. It emits the byte and jumps back to the root.

Decoding the first four symbols - follow bits down the tree, emit at a leaf, jump back to the root

Bits read	Path through tree (0 = left, 1 = right)	Result
0	root --0--> leaf A	A
1 1 0	root --1--> node 6 --1--> node 4 --0--> leaf B	B
1 1 1	root --1--> node 6 --1--> node 4 --1--> leaf R	R
0	root --0--> leaf A	A

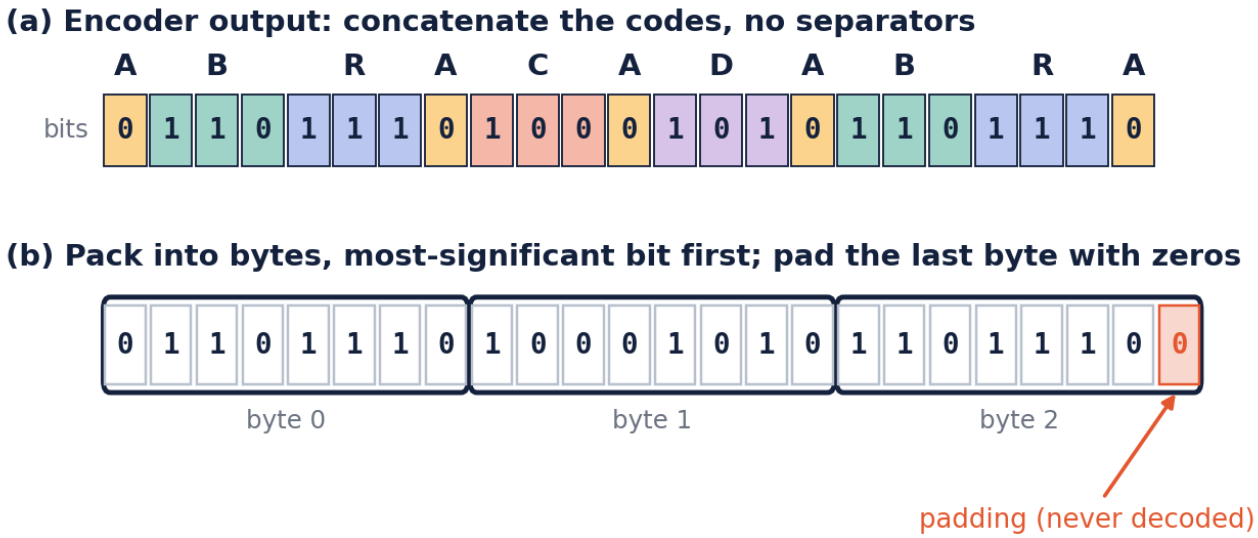
No bit is wasted marking where one symbol ends: reaching a leaf is the boundary. That is the prefix property.

Figure 5. Decoding the start of the stream 0 110 111 0 … Reaching a leaf is the boundary between symbols; no length field or delimiter is stored.

> **INTUITION**
> A **non-prefix** set such as {0, 01, 11} fails: after reading a 0 the decoder cannot tell whether the symbol is 'a 0' or the start of '01'. Prefix-freeness is what turns a stream of bits into an unambiguous stream of symbols.

4.2 Packing bits into bytes

Files are made of whole bytes, but Huffman codes are 1, 3, 7 … bits long. **BitWriter** therefore keeps a small buffer, appends code bits **most-significant bit first**, and emits a byte each time eight bits have accumulated. **BitReader** uses the identical order to take them apart again. If the final byte is only partly filled, its remaining positions are padded with zero bits.



Stored payload bytes: `0x6E` `0x8A` `0xDC`

The decoder stops after 11 symbols (the stored original size), so the padding bit is ignored.

Figure 6. The 23-bit message becomes three bytes, 0x6E 0x8A 0xDC. The single padding bit in the last byte exists only because 23 is not a multiple of 8. *Illustration using the example tree.*

The padding bit is harmless *because the header stores the original file size*. The decoder counts the symbols it has produced and stops exactly at that number, so trailing padding is never mistaken for data. The same stored size gives a

simple integrity check: if the bit stream ends before the expected number of bytes has been rebuilt, the file is truncated or invalid and decoding must fail rather than silently produce short output.

5 System architecture

The program is split into small components with one responsibility each. **Compressor** and **Decompressor** orchestrate; the lower blocks do the work. Two components, highlighted in amber in Figure 7, are shared by both directions, and they are where most compatibility bugs would live.

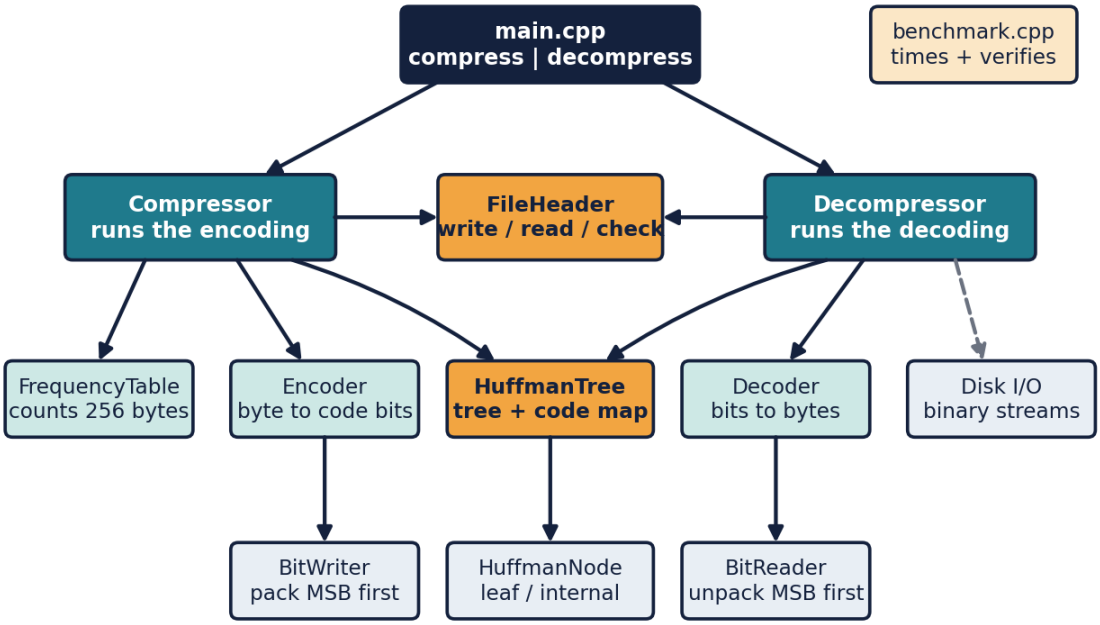


Figure 7. Component structure. Arrows read 'uses'. *The Disk I/O link is schematic; both orchestrators read and write binary streams.*

Table 2. Component responsibilities, from the project's technical documentation.

Component	Responsibility (recorded)
`FrequencyTable`	Counts the frequency of each byte value (0–255) and tracks the input size.
`HuffmanNode`	Represents a leaf or an internal node of the tree.
`HuffmanTree`	Builds the tree with a min-priority queue and produces byte-to-code mappings.
`BitWriter / BitReader`	Pack code bits into output bytes, most-significant bit first / read them back in the same order.
`Encoder / Decoder`	Write the Huffman code sequence for the source / traverse the tree from encoded bits and emit original bytes.
`FileHeader`	Serialises and deserialises format metadata and validates header information.
`Compressor`	Coordinates frequency counting, tree construction, header writing and encoding.
`Decompressor`	Reads the header, rebuilds the tree, selects the decoding path and writes the restored file.
`main.cpp`	Parses the command-line operation and the input and output paths.
`benchmark.cpp`	Measures elapsed time and output size and checks the restored data.

6 The compressed file format

The decoder cannot guess the tree, so the file must carry the information needed to rebuild it. FileCompressor stores the **frequency table** of all 256 byte values; because tree construction is deterministic, the decoder runs the same algorithm on the same frequencies and obtains an identical tree. This avoids storing every code explicitly.

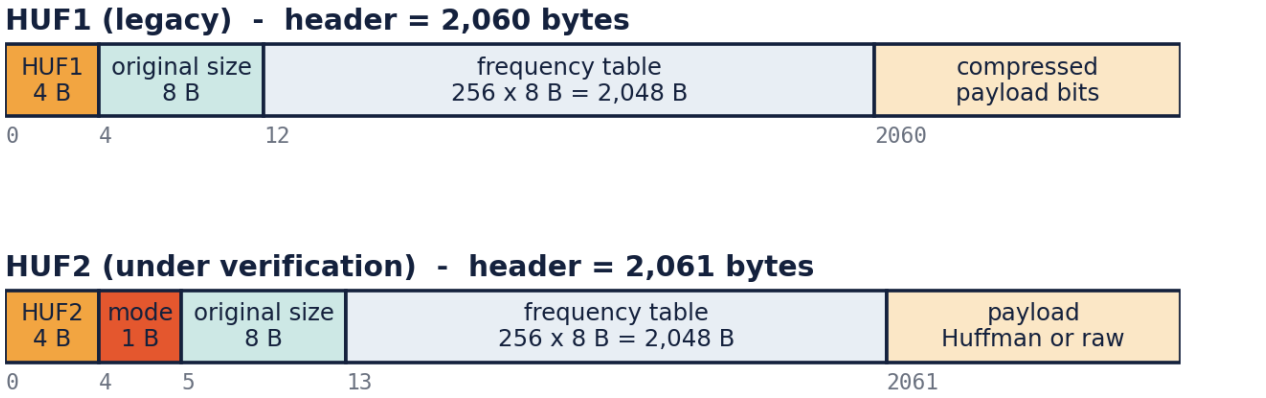


Figure 8. Layout of the two formats (recorded sizes). The frequency table makes up 2,048 of the header's 2,060 (or 2,061) bytes.

Table 3. Header fields of the legacy and newer formats (recorded).

Field	HUF1	HUF2
Magic bytes	`HUF1` · 4 B	`HUF2` · 4 B
Mode	–	1 B (0 = raw, 1 = Huffman)
Original file size	8 B	8 B
Frequencies (256 × 8 B)	2,048 B	2,048 B (zero-filled in raw mode)
Header total	**2,060 B**	**2,061 B**
Status	Used for the working round-trip implementation	Under verification (see Section 10)

WATCH OUT

Portability. The current implementation writes 64-bit integers using their *in-memory* representation. On a little-endian machine such as typical x86 PCs, the size 11 is stored as `0B 00 00 00 00 00 00 00`; a big-endian machine would write `00` … `00 0B`. A file written on one kind of machine could therefore be misread on another. A future format revision should define a fixed byte order and version it. *(The byte sequences shown are an illustration of the concept.)*

NOTE

Header cost. A fixed 2,060-byte header is negligible on a 100 MiB file (about 0.002%) but dominates tiny files: a 100-byte file cannot come out smaller than 2,060 bytes. Section 9 quantifies this.

7 Compression and decompression pipelines

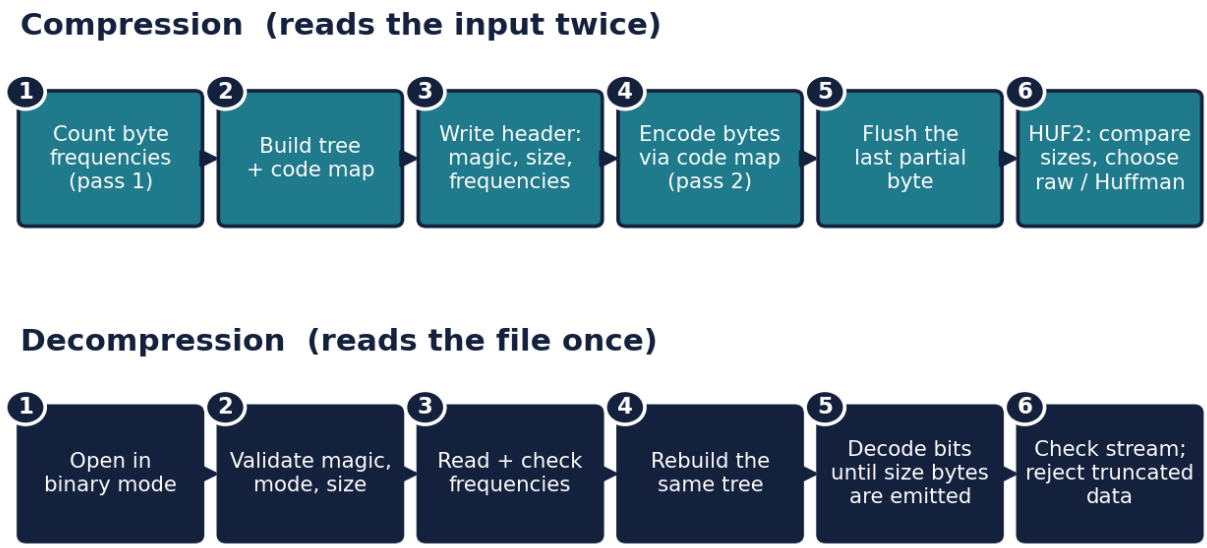


Figure 9. End-to-end flow. Compression needs two passes over the input (count, then encode); decompression is a single pass over the compressed file.

7.1 Compression

- **Validate** that the input and output paths differ, so the source can never be overwritten while it is being read.
- **Pass 1**: read the file once and build the 256-entry frequency table.
- **Build** the Huffman tree and the byte-to-code map.
- **Write the header**: magic bytes, original size, frequencies.
- **Pass 2**: read the file again; look up each byte's code and feed its bits to BitWriter.
- **Flush** the final partial byte (zero padded).
- **HUF2 only**: compare the Huffman output size with the expected raw-mode size and pick the smaller mode. Because raw mode still carries the full fixed header, even it can exceed the source size.

7.2 Decompression

- Open the compressed file in **binary mode**; validate the magic bytes, read the mode and the original size.
- Read and validate the frequencies (Huffman mode) and rebuild the tree.
- Decode bit by bit and emit bytes until the original size is reached; in raw mode copy exactly the declared number of bytes.
- Check the stream state and reject truncated data.

7.3 Defensive behaviour

The documentation lists the failure cases decompression should guard against. They are the difference between a program that works on good input and one that is safe on any input:

Table 4. Edge cases and defensive checks (derived from the documented requirements).

Situation	Expected behaviour	Reason
Empty input	Original size 0; output is an empty file, no symbol is decoded.	A zero-leaf tree has nothing to traverse.
Single distinct byte	Symbol gets 1-bit code `0`; decoder handles the one-leaf tree explicitly.	No branch decisions exist in a one-leaf tree.
Output path = input path	Refuse and report an error.	Writing output would destroy the source.
Invalid magic or mode	Reject with a useful error.	Not a file this program produced, or a newer format.
Inconsistent frequency totals	Reject; do not build a tree.	Total should equal the stored original size.
Incomplete header	Reject.	Metadata is unreadable.
Truncated payload	Reject; never silently emit short output.	Bit stream ended before the original size was rebuilt.

8 Complexity analysis

Let **n** be the number of input bytes, **k ≤ 256** the number of distinct byte values and **b** the number of encoded bits.

Table 5. Complexity of each phase (recorded analysis).

Phase	Time	Extra space	Comment
Frequency counting	`O(n)`	O(256) = O(1)	Every byte is read once into a fixed table.
Tree construction	`O(k log k)`	O(k)	At most 256 leaves, so bounded by a small constant.
Encoding	`O(n + b)`	code table + small bit buffer	One lookup per byte, one write per output bit.
Decoding	`O(b + n)`	tree + small bit buffer	One tree edge per encoded bit, one emit per symbol.

The expensive parts all scale linearly with the file size; the tree, which feels like the 'clever' part, is negligible because the alphabet has at most 256 symbols. That is also why, in practice, wall-clock time is governed by storage speed, buffering and the implementation of the bit-level loop, which is where optimisation effort should go.

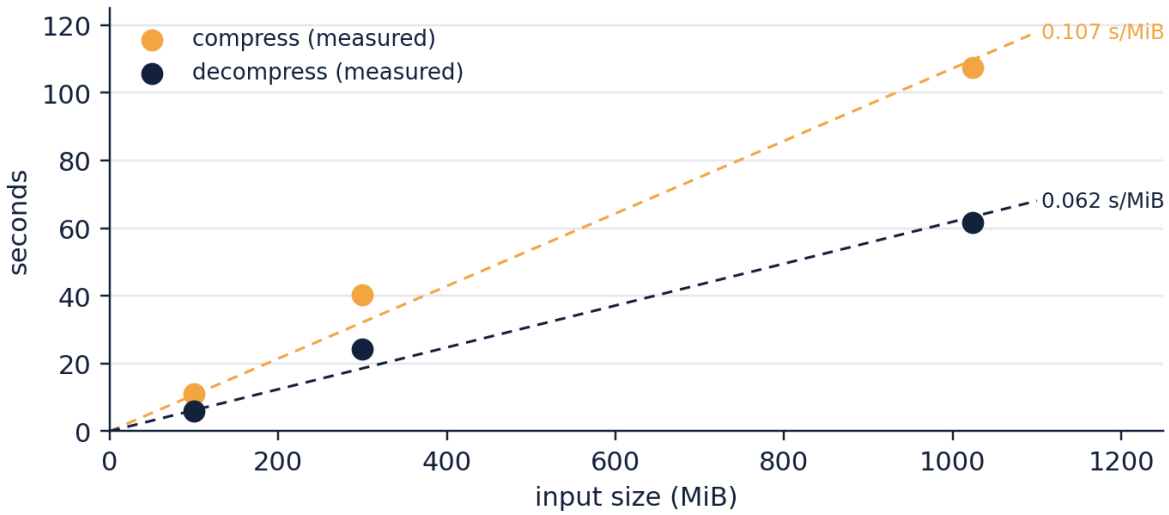


Figure 10. Recorded timings against input size. The dashed lines are least-squares fits through the origin (*derived* here): about 0.11 s per MiB to compress and 0.06 s per MiB to decompress, i.e. close to linear scaling.

9 Empirical results

The project recorded three local runs. These describe the tested files and machine only; they are not guarantees for other inputs.

Table 6. Recorded benchmark results.

Input	Compressed	Ratio	Saved	Compress	Decompress
100 MiB	`59,712,585 B`	`56.95%`	`43.05%`	`11.09 s`	`5.85 s`
~300 MiB	`191,185,490 B`	`60.77%`	`39.23%`	`40.29 s`	`24.33 s`
~1 GiB	`653,627,120 B`	`60.87%`	`39.13%`	`107.39 s`	`61.72 s`

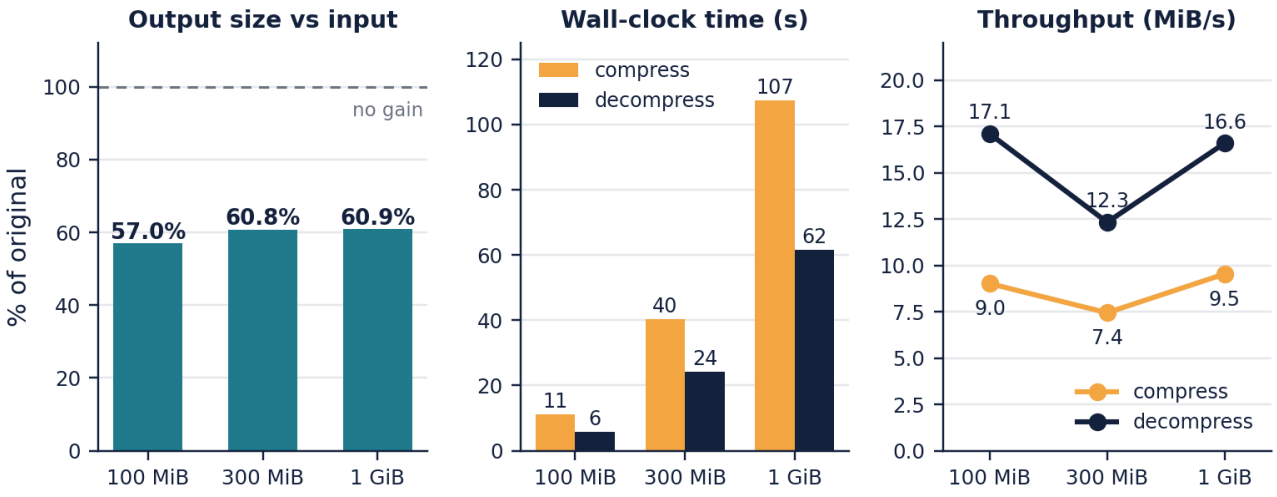


Figure 11. Benchmark summary. The throughput panel is *derived*: nominal size ÷ recorded time, taking '~300 MiB' as 300 MiB and '~1 GiB' as 1,024 MiB (consistent with the recorded ratios).

What the numbers say

- **Substantial savings that level off.** The ratio stays between 57% and 61%, equivalent to roughly 4.6 to 4.9 bits per byte, so the tested data is clearly uneven but far from trivially repetitive.
- **Decompression is faster than compression**, about 1.7–1.9× in these runs. This is consistent with compression needing two passes over the input while decompression is one pass, but the runs alone do not prove the cause.

- **Throughput is roughly steady**, about 7–10 MiB/s compressing and 12–17 MiB/s decompressing, which supports the linear-time analysis. The report's own recommendation is to profile and improve buffered bit I/O, since this is a long way from storage limits.
- **Per-file overhead is invisible at this scale**: the 2,060-byte header is about 0.002% of 100 MiB.

Where Huffman stops helping: the header-overhead model

With a fixed header of H_0 = 2,060 bytes and a payload that shrinks to a fraction r of the input, the output size is $H_0 + r \cdot n$. The file gets smaller only if $n > H_0 / (1 - r)$. For text-like data with $r \approx 0.6$ that is about 5.2 KB; for incompressible data ($r \approx 1$) it never breaks even.

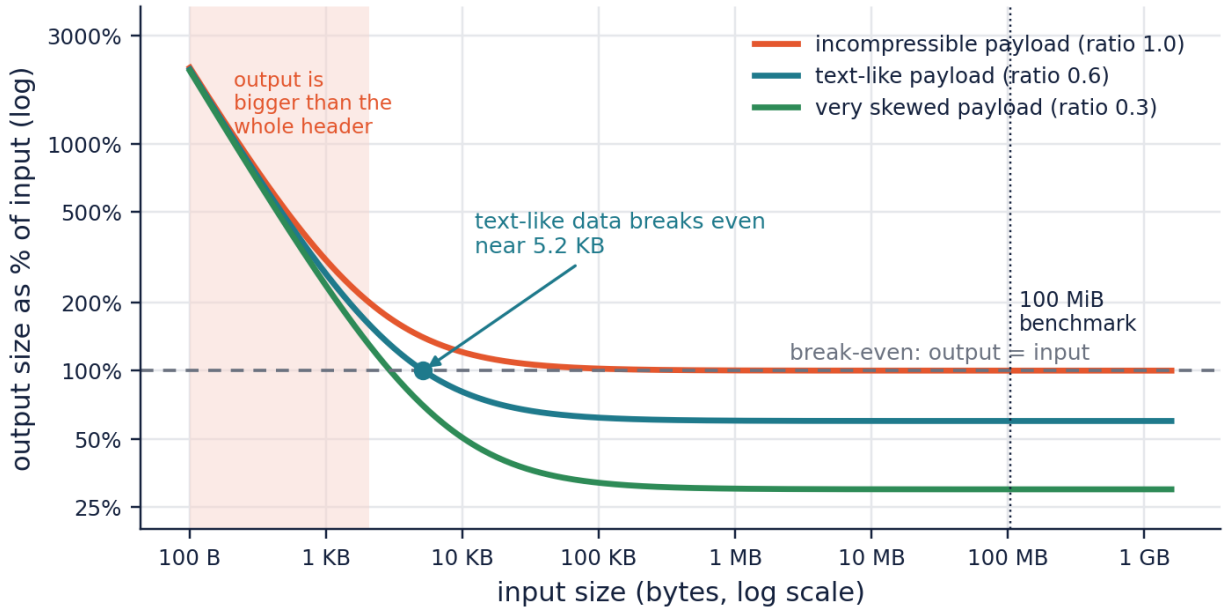


Figure 12. *Analytical model, not a measurement.* Output size as a percentage of input size for a 2,060-byte header and three payload ratios. Small files end up larger than they started; large files converge to the payload ratio.

KEY IDEA

This is why the HUF2 design adds a **raw mode**: when Huffman does not shrink the payload, store the bytes unchanged. It is a good idea, but with the full fixed header retained, raw output is still *input + 2,061 bytes*, so it limits the damage and does not remove it.

10 Verification status and the open issue

A compressor is only as good as its round trip: *decompress(compress(x))* must equal *x* for every *x*. The technical documentation is explicit about what is and is not established, and this report keeps the same discipline.

Table 7. Verification status. Based strictly on the project's README and technical documentation.

Component	Status	What the documentation says
HUF1 round trip	WORKING	Original format used for the working round-trip implementation.
HUF2 Huffman mode (1)	NOT CONFIRMED	No separate pass result is recorded; treat as unverified until the checklist passes.
HUF2 raw mode (0)	KNOWN ISSUE	A byte mismatch appeared in a random-binary round-trip test. Do not call it stable.
Benchmark restore check	FEATURE	The benchmark program checks restored data; per-run pass results are not recorded in the docs.

> **WATCH OUT**
> **Do not publish a claim that all modes pass correctness tests** until the HUF2 raw-mode mismatch is resolved, a fresh clean rebuild passes, and repeated hash or byte comparisons succeed. This restriction appears in both project documents.

10.1 Release checklist (all items currently open)

	Test or task
[ ]	Rebuild the CLI from a clean state.
[ ]	Round-trip: empty, one-symbol, text, all-256-byte-values, random-binary (several MiB) and large inputs.
[ ]	Compare every restored file with its source using `fc.exe /b` or SHA-256.
[ ]	Malformed and truncated files fail with a useful error instead of silently producing output.
[ ]	Input and output paths cannot be identical.
[ ]	Resolve the HUF2 raw-mode mismatch before marking HUF2 stable.
[ ]	Generated binaries, temp files, test data and outputs are ignored by Git.
[ ]	Update the README to match the actually tested behaviour.

```
# byte-for-byte comparison (Windows)
fc.exe /b .\input.bin .\output\restored.bin

# hash comparison
Get-FileHash .\input.bin -Algorithm SHA256
Get-FileHash .\output\restored.bin -Algorithm SHA256
```

10.2 Suggested ways to hunt the raw-mode mismatch

The following are *hypotheses to test*, ordered from cheapest to most involved. They are suggestions from this report and are not conclusions about your code.

Table 8. Debugging hypotheses for the HUF2 raw-mode mismatch (suggestions).

#	Check	What it would explain
1	Rebuild everything from a clean state (the documentation's own first step).	Stale object files mixing HUF1 and HUF2 code.
2	Run the byte compare and note the **first differing offset**. Offset 0–12 points at the header; a mid-file offset points at the payload loop; a difference only at the end points at size or padding handling.	Tells you which part of the pipeline to look at before reading any code.
3	Confirm every file stream is opened with `std::ios::binary`, on input and output.	On Windows, text-mode streams translate line endings and treat 0x1A specially; random binary data hits those bytes, plain text mostly does not.
4	Check the header is written field by field, not by writing a struct to disk.	Compiler padding after the 1-byte mode field would make the real header differ from 2,061 bytes.
5	Make sure raw mode copies bytes with unformatted reads (`read` / `write`), not formatted extraction.	Formatted reads can skip whitespace-valued bytes, which are common in random data.
6	Verify raw mode copies exactly the declared original size, including the final partial buffer.	A short or long last chunk.

11 Limitations and the road ahead

Table 9. Documented limitations and the improvements the project proposes.

Limitation (recorded)	Why it matters	Direction
Fixed 256-entry frequency table in every header.	Dominates the size of small files (Figure 12).	Store less metadata (see the canonical-code idea below); omit the table for raw data.
Integers written in native byte order.	File may not read correctly on another architecture.	Define a fixed byte order and version the format.
HUF2 raw fallback unverified.	Cannot yet be called stable.	Pass the random-binary round trip and the full checklist.
Large-file speed.	7–10 MiB/s compression leaves room.	Faster buffered bit I/O; profile first.
Limited validation.	Corrupt files could produce wrong output.	Payload-length checks, malformed-tree checks, a checksum.

An idea beyond the current plan: canonical Huffman codes

(Suggestion; not part of the current project documentation.) The decoder does not need the frequencies themselves, only the **length** of each symbol's code. If codes of equal length are assigned in a fixed order (alphabetical by byte value), the tree can be rebuilt from 256 one-byte lengths. That would shrink the metadata from 2,048 bytes to 256, bringing the header to about 4 + 8 + 256 = 268 bytes and moving the text-like break-even point from about 5.2 KB down to roughly 0.7 KB (268 ÷ 0.4). It would also remove any dependence on the tie-breaking rule, since the lengths pin the code book down completely. It would be a new format version, so it fits naturally alongside the byte-order change already on the list.

> INTUITION
> **Design principle to carry away:** a file format is a contract between an encoder and a decoder that may run years apart, on different machines. Everything the decoder relies on (byte order, tie-breaking, padding, version number) must be written down in the format, not left to chance.

12 Study guide

Ten things to remember

1. Lossless compression exploits **uneven symbol frequencies**; it cannot shrink data that is already uniform.
2. Huffman gives frequent bytes **short** codes and rare bytes **long** codes.
3. The tree is built by repeatedly **merging the two lightest nodes** (a greedy algorithm, optimal for whole-bit codes).
4. A code is the **left/right path** to a leaf: left = 0, right = 1.
5. Symbols live only at leaves, so no code is a prefix of another and **no separators** are needed.
6. Bits are packed **MSB first**; the last byte is zero-padded and ignored thanks to the stored original size.
7. The header stores the **frequency table** so the decoder can rebuild the identical tree (2,060 bytes in HUF1).
8. Special cases: **empty input** and **one distinct byte** (1-bit code 0) must be handled on purpose.
9. Everything is **O(n)** in the file size; the tree is O(k log k) with $k \le 256$ and effectively free.
10. Trust results only after a **byte-for-byte round trip**; HUF2 raw mode has not earned that trust yet.

Exercises

#	Exercise
1	Build a Huffman tree for `MISSISSIPPI` (M=1, I=4, S=4, P=2). How many bits does the encoded message need? How many bytes is that after packing?
2	Is the code set {0, 01, 11} a valid prefix code? Explain what goes wrong for a decoder.
3	A file contains 1,000,000 copies of the same byte. With a 2,060-byte header and a 1-bit code, how large is the HUF1 output?
4	Using the model of Section 9, at what input size does a text-like file (ratio 0.6) first come out smaller with a 2,060-byte header?
5	Why must the decoder know the original file size, even though every code is prefix-free?
6	Why does the benchmark show decompression running faster than compression?

Answers

#	Answer
1	Merge M(1)+P(2)=3; then 3+I(4)=7; then S(4)+7=11. Code lengths: S=1, I=2, M=3, P=3. Bits = 4×1 + 4×2 + 1×3 + 2×3 = **21 bits** (versus 88 at 8 bits per symbol), which packs into **3 bytes**.
2	No. The code 0 is a prefix of 01, so after reading a 0 the decoder cannot tell whether a symbol ended or continues. Decoding becomes ambiguous.
3	1,000,000 bits = 125,000 bytes of payload, plus the header: **127,060 bytes** (about 12.7% of the original).
4	Solve 2,060 + 0.6·n < n, so n > 2,060 ÷ 0.4 = **5,150 bytes** (about 5.2 KB).
5	The last byte may contain padding bits. Without a stored count the decoder could interpret those zeros as extra symbols; the size tells it exactly when to stop. It also enables truncation detection.
6	A plausible reason is that compression reads the input twice (count, then encode) while decompression makes one pass; the recorded timings are consistent with this but do not isolate the cause.

Glossary

Term	Meaning
Lossless	Decompression reproduces the original bytes exactly.
Entropy	Average information per symbol in bits; the floor for whole-bit symbol codes.
Prefix code	A code set in which no code is the start of another, so it decodes without delimiters.
Huffman tree	Binary tree whose leaves are symbols and whose root-to-leaf paths are their codes.
Min-priority queue	Structure that always yields the smallest-frequency node first.
MSB first	Writing the most significant (leftmost) bit of a byte before the others.
Padding	Extra zero bits that fill the last output byte; never decoded.
Magic bytes	Fixed leading bytes (HUF1, HUF2) that identify the file format.
Round trip	Compress then decompress; the result must equal the original.
Endianness	Whether the low or the high byte of a multi-byte integer is stored first.
Throughput	Data processed per second, such as MiB/s.

A Appendix: pseudocode and commands

A.1 Conceptual pseudocode

Language-neutral sketch of the algorithms described above; not a copy of the project's source.

```
compress(input, output):
    require input_path != output_path
    freq[0..255] = count each byte of input           # pass 1
    tree = build_huffman_tree(freq)                   # min-priority queue
    codes = assign_codes(tree)                        # left = 0, right = 1
    write_header(magic, size(input), freq)
    for each byte x in input:                         # pass 2
        bitwriter.write_bits(codes[x])                # MSB first
    bitwriter.flush()                                 # zero-pad last byte

decompress(input, output):
    magic, mode, size, freq = read_and_validate_header(input)
    if size == 0: create empty output; return
    tree = build_huffman_tree(freq)                   # same algorithm
    node = tree.root;  emitted = 0
    while emitted < size:
        bit = bitreader.read_bit()  or  fail("truncated")
        node = node.child[bit]
        if node is leaf:
            write(output, node.byte);  emitted += 1;  node = tree.root
```

A.2 Build and run (from the project README, PowerShell)

```
g++ -std=c++17 main.cpp src/BitWriter.cpp src/FrequencyTable.cpp src/HuffmanNode.cpp \
    src/HuffmanTree.cpp src/Encoder.cpp src/FileHeader.cpp src/Compressor.cpp \
    src/BitReader.cpp src/Decoder.cpp src/Decompressor.cpp -Iinclude -o compressor

.\compressor.exe compress   .\input.txt        .\output\input.huf
.\compressor.exe decompress .\output\input.huf .\output\restored.txt
.\benchmark.exe
```

References

[1] D. A. Huffman, 'A Method for the Construction of Minimum-Redundancy Codes,' *Proceedings of the IRE*, 40(9), 1098–1101, 1952.

[2] C. E. Shannon, 'A Mathematical Theory of Communication,' *Bell System Technical Journal*, 27, 379–423 and 623–656, 1948.

[3] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006.

[4] FileCompressor project: README and Technical Documentation (provided).